Day 40: Weight Initialization and Batch Normalization (Beginner Edition)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 6, Day 5

Every network from Day 36 through Day 39 quietly relied on one fixed rule for starting weights — randn(...) * 0.5 — which happened to work fine on small, shallow (2-layer) networks. Today asks what happens once a network actually gets deep, and shows — with a real, rerun 6-hidden-layer network — that the SCALE of the starting random numbers, chosen systematically rather than guessed, is the difference between a network that trains and one that never learns anything at all.

0. Where Today Fits: Why Depth Breaks 'Just Pick Small Random Weights'

Start here (no prior knowledge assumed): Every network built so far had exactly one hidden layer between input and output — shallow enough that almost any reasonable initialization scale worked fine. Today's network has SIX hidden layers. At that depth, a poorly chosen initialization scale doesn't just slow training down — verified below, it can freeze a network so completely that it never learns anything at all, for the entire training run.

Term refresher — Fan-in: The FAN-IN of a layer is simply the number of inputs feeding into it — for a layer mapping 32 units to 32 units, the fan-in is 32. Both Xavier and He initialization scale their random weights based directly on this number, which is why the same formula behaves differently at different layer widths.

Watch: L11.6 Xavier Glorot and Kaiming He Initialization — Sebastian Raschka — A focused lecture-style walkthrough of exactly today's two systematic initialization strategies, from the same STAT 453 deep learning course series.

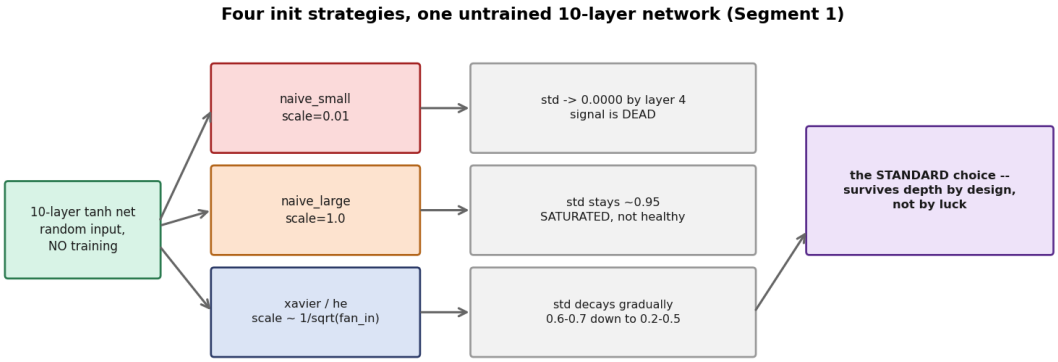
1. Signal Propagation — Watching an Untrained Deep Network Breathe (or Suffocate)

Start here (no prior knowledge assumed): Before any training happens, it's possible to ask a simpler question: if random data is pushed through a deep network with NO learning at all, does the signal survive 10 layers deep, or does it die out (or blow up) purely from the choice of starting weight scale? This section answers that question directly, by measuring the standard deviation of the activations at every layer.

Topic specification: propagate_signal() pushes 200 random points through 10 layers of a 64-unit tanh network, under 4 different initialization scales, recording each layer's output standard deviation. No weights are ever updated — this is a pure diagnostic of what the STARTING scale alone does to a signal as depth increases.

Term refresher — Saturation: SATURATION means an activation function's output is pinned near its extreme values (for tanh, near +1 or -1), where its derivative is nearly zero. A saturated network isn't dead the way a collapsed one is, but its gradients vanish almost as badly — a high activation std is not automatically a healthy sign.

How it works, visually:



The four init strategies compared side by side, and what happens to a random signal under each one after 10 layers with no training.

Explanation: Each layer's pre-activation z is a weighted sum of many inputs — as more inputs get summed, the natural spread (variance) of that sum grows unless the individual weights are scaled down to compensate. Get the scale wrong in either direction and depth compounds the error: too small and the signal shrinks every layer until it vanishes; too large and it grows (or saturates) every layer instead.

Real-world scenario: This is exactly the failure mode that historically limited how deep networks could practically go before Xavier and He initialization were introduced — architectures with even 10-20 layers were difficult to train reliably with naive random initialization, regardless of how much data or compute was available, because the SIGNAL itself couldn't survive the forward pass.

Mathematics:

MATH

For one layer: $z = a \times W + b$, with W drawn as randn(...) × scale.
Treating each input as roughly independent: $Var(z) \approx n\_in \times Var(a) \times scale^2$
scale = 0.01 (naive_small): Var(z) shrinks by a large factor every layer → signal vanishes.
scale = 1.0 (naive_large): Var(z) grows by a large factor every layer → tanh saturates near ±1.

Implementation:

```
def propagate_signal(n_layers, n_units, strategy, use_bn=False, n_samples=200, seed=0):
    rng = np.random.RandomState(seed)
    a = rng.randn(n_samples, n_units)
    stds = []
```

```
    for _ in range(n_layers):
        scale = init_scale(strategy, n_units)
        W = rng.randn(n_units, n_units) * scale
        b = np.zeros(n_units)
        z = a @ W + b
        a = tanh(z)
        stds.append(a.std())
    return stds
```

Actual output:

```
naive_small    layer stds: ['0.0776', '0.0062', '0.0005', '0.0000', '0.0000', ... '0.0000']
naive_large    layer stds: ['0.9457', '0.9455', '0.9455', '0.9487', '0.9492', ... '0.9462']
xavier         layer stds: ['0.6196', '0.4830', '0.3995', '0.3575', '0.3311', ... '0.2190']
he             layer stds: ['0.7137', '0.6316', '0.5890', '0.5781', '0.5773', ... '0.5426']
```

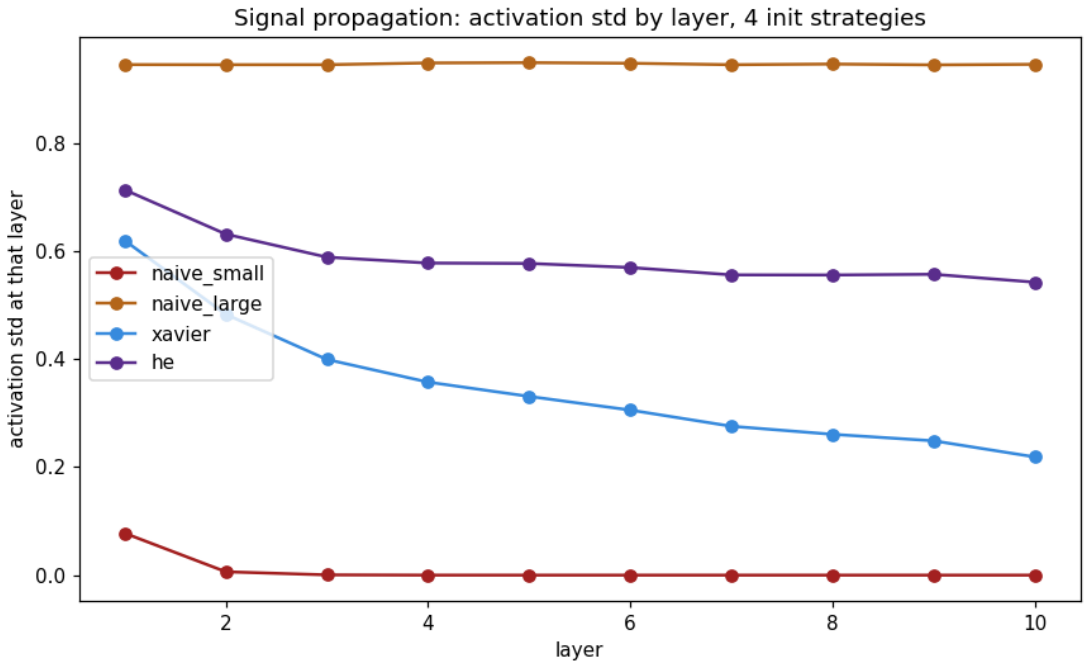
Actual output from this lesson's run (10 layers, 64 units, tanh, no training).

Line-by-line explanation

• Verified: naive_small's std hits exactly 0.0000 by layer 4 and stays there through layer 10 — the signal is completely dead, not just small. naive_large stays pinned around 0.945-0.949 at every layer — high, but that's saturation (tanh near its flat extremes), not health.

• xavier and he both decay gradually and smoothly (0.62 to 0.22, and 0.71 to 0.54 respectively) — real signal loss over 10 layers, but nowhere near naive_small's total collapse or naive_large's saturation.

How it works, visually:



REAL GRAPH (script's own output) — activation std by layer, all four strategies plotted together, from this lesson's actual run.

Why choose this? Measuring this directly, with no training involved at all, isolates initialization scale as the ONLY variable — an honest, mechanical demonstration rather than a claim taken on faith.

Why NOT this (tradeoffs)? This diagnostic uses tanh specifically and 10 layers of 64 units specifically — the exact numbers would shift with a different activation function, width, or depth, though the underlying mechanism (variance compounding across layers) generalizes.

2. Xavier / Glorot Initialization — Solving for the Scale That Preserves Variance

Start here (no prior knowledge assumed): Rather than guessing a fixed scale like 0.01 or 1.0, Xavier (also called Glorot) initialization SOLVES for the scale that keeps a layer's output variance equal to its input's variance — so the signal neither shrinks nor grows, layer after layer, purely from initialization.

Topic specification: Xavier sets scale = sqrt(1 / n_in), where n_in is the layer's fan-in. This is derived directly by requiring the layer's output variance to equal its input variance, and solving for the one unknown (scale) that makes that true.

Explanation: This is why Xavier's decay in Topic 1's graph (0.62 down to 0.22) is real but far more gradual than naive_small's — the formula doesn't perfectly preserve variance forever across 10 tanh layers (tanh's nonlinearity itself causes some genuine signal loss), but it removes the CATASTROPHIC compounding that an arbitrary fixed scale causes.

Real-world scenario: Xavier initialization is the standard, well-established default for networks using symmetric, saturating activations like tanh or sigmoid — it was specifically introduced to solve exactly the vanishing/exploding signal problem demonstrated numerically in Topic 1.

Mathematics:

MATH

Goal: choose scale so that Var(z) = Var(a) (output variance equals input variance).
$Var(z) \approx n\_in \times Var(a) \times scale^2$ *(from Topic 1's math box)*
$Setting\ Var(z) = Var(a) \rightarrow n\_in \times scale^2 = 1 \rightarrow scale^2 = 1 / n\_in$
$\mathbf{scale = sqrt(1 / n\_in)}$

Implementation:

```
def init_scale(strategy, n_in):
    if strategy == "xavier":
        return np.sqrt(1.0 / n_in)
    # ... naive_small, naive_large, he handled by the other branches
```

Actual output:

```
xavier         layer stds: ['0.6196', '0.4830', '0.3995', '0.3575', '0.3311',
                            '0.3058', '0.2758', '0.2609', '0.2488', '0.2190']
```

Actual output from this lesson's run — same numbers shown in Topic 1, isolated here.

Why choose this? Deriving the scale from a variance-preservation requirement, rather than guessing, means the same formula automatically adapts to any layer width — a 64-unit layer and a 256-unit layer each get their own correctly-scaled initialization with zero manual tuning.

Why NOT this (tradeoffs)? Xavier's derivation assumes a roughly linear, symmetric activation around zero — it's a well-matched default for tanh/sigmoid, but (as Topic 3 covers next) a different derivation is needed for ReLU-family activations, which are not symmetric around zero.

3. He / Kaiming Initialization — Xavier's ReLU-Oriented Cousin

Start here (no prior knowledge assumed): ReLU activations output exactly 0 for roughly half their inputs (anything negative) — this breaks Xavier's symmetric-around-zero assumption. He (also called Kaiming) initialization adjusts Xavier's formula by a factor of 2 to compensate for this.

Topic specification: He sets scale = sqrt(2 / n_in) — identical in form to Xavier, but with numerator 2 instead of 1, specifically designed for layers followed by a ReLU-family activation.

Explanation: Because ReLU zeroes out roughly half its inputs, the variance of its OUTPUT is roughly half the variance of its INPUT, even under otherwise perfect initialization. Doubling the target variance (using 2 instead of 1 in the numerator) exactly compensates for that expected 50% loss, restoring the same variance-preservation property Xavier provides for tanh/sigmoid.

Real-world scenario: Almost every modern deep convolutional or fully-connected network using ReLU (or a ReLU variant like LeakyReLU) defaults to He initialization for exactly this reason — it's the systematic match for the activation function's own asymmetric behavior, not an arbitrary alternative to Xavier.

Mathematics:

MATH

ReLU zeroes roughly half its inputs, so for ReLU output r = max(0, z):
Var(r) ≈ 0.5 × Var(z) (only the positive half contributes variance)
To preserve variance despite this loss, require Var(z) = 2 × Var(a) instead of Var(z) = Var(a)
$n\_in \times scale^2 = 2 \rightarrow scale^2 = 2 / n\_in \rightarrow$ ***scale = sqrt(2 / n_in)***

Implementation:

```
def init_scale(strategy, n_in):
    if strategy == "he":
        return np.sqrt(2.0 / n_in)
    # ... naive_small, naive_large, xavier handled by the other branches
```

Actual output:

```
he             layer stds: ['0.7137', '0.6316', '0.5890', '0.5781', '0.5773',
                            '0.5699', '0.5562', '0.5560', '0.5573', '0.5426']
```

Actual output from this lesson's run — same numbers shown in Topic 1, isolated here.

Line-by-line explanation

• Verified: he's activations settle noticeably higher than xavier's at every layer (0.54 vs 0.22 by layer 10) — this script uses tanh throughout (not ReLU) purely as a consistent diagnostic signal, so he's extra factor of 2 shows up here as simply a larger, less-decayed std, not as its intended ReLU-specific correction.

Why choose this? He initialization is the standard, well-matched default for ReLU-family networks — using Xavier instead on a ReLU network under-scales the weights relative to what the activation's own zeroing behavior requires.

Why NOT this (tradeoffs)? Using He initialization on a tanh or sigmoid network (as this diagnostic technically does, for direct comparability with the other three strategies) over-scales relative to what tanh/sigmoid's symmetric behavior calls for — the formula should match the activation it precedes, not be picked independently of it.

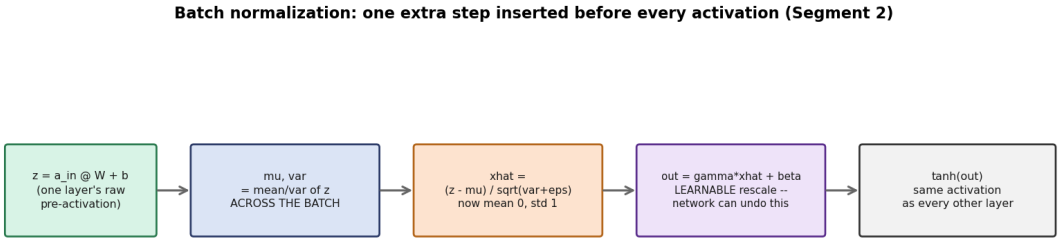
4. Batch Normalization — The Forward-Pass Mechanism

Start here (no prior knowledge assumed): Instead of relying on getting the initialization SCALE exactly right, batch normalization takes a completely different approach: it directly normalizes each layer's pre-activation values, every forward pass, regardless of what scale the weights started at.

Topic specification: For each layer, batch norm computes the mean and variance of the pre-activation z ACROSS THE CURRENT BATCH, normalizes z to have mean 0 and std 1, then applies a learnable rescale (gamma) and shift (beta) — allowing the network to undo the normalization if that turns out to be better for a specific layer.

Term refresher — Gamma and beta: GAMMA and BETA are learnable parameters, one pair per normalized layer, initialized to gamma=1 and beta=0 (the identity transform). They're updated by gradient descent just like weights and biases — giving the network the ability to learn its own preferred scale and shift for each layer's normalized output, rather than being forced into exactly mean-0, std-1 forever.

How it works, visually:



The five-step pipeline batch normalization inserts before every hidden layer's activation.

Explanation: Because normalization happens FRESH every forward pass, using the CURRENT batch's own statistics, it doesn't matter whether the incoming z values started out tiny (from a collapsed init) or huge (from a saturated init) — after normalization, they're rescaled to the same stable range every single time, regardless of what came before.

Real-world scenario: This is why batch normalization is described as making training 'robust to initialization' — rather than requiring the engineer to carefully choose scale = sqrt(1/n_in) or sqrt(2/n_in) up front, batch norm enforces a healthy signal range directly, layer by layer, automatically.

Mathematics:

MATH

mu = mean(z, across the batch) var = variance(z, across the batch)

xhat = (z - mu) / sqrt(var + eps) (now mean 0, std 1, eps prevents /0)

z_bn* = *gamma* × *xhat* + *beta *(learnable rescale/shift, then passed to tanh)*

Implementation:

```
if self.use_bn:
    mu = z.mean(axis=0, keepdims=True)
    var = z.var(axis=0, keepdims=True)
    xhat = (z - mu) / np.sqrt(var + 1e-5)
    z = self.gammas[i] * xhat + self.betas[i]
a_out = tanh(z)
```

Actual output:

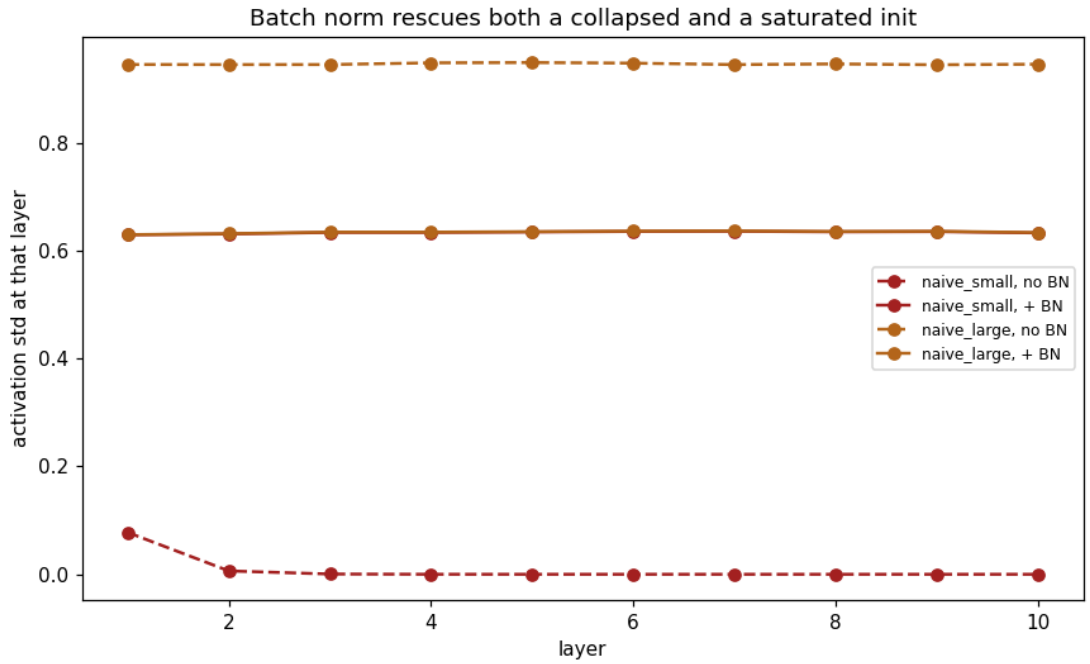
```
naive_small WITHOUT batch norm: ['0.0776', '0.0062', ... '0.0000', '0.0000']
naive_small WITH    batch norm: ['0.6297', '0.6313', ... '0.6357', '0.6333']

naive_large WITHOUT batch norm: ['0.9457', '0.9455', ... '0.9451', '0.9462']
naive_large WITH    batch norm: ['0.6300', '0.6319', ... '0.6363', '0.6339']
```

Actual output from this lesson's run — both a collapsed and a saturated init, before/after batch norm.

Line-by-line explanation

• Verified: with batch norm, BOTH naive_small (previously dead at 0.0000) and naive_large (previously saturated at ~0.95) settle to the same stable ~0.63 std at every single layer — batch norm made the signal's health completely independent of which bad initialization was used.

How it works, visually:



REAL GRAPH (script's own output) — the same collapsed and saturated inits, rescued to nearly identical stable curves once batch norm is applied.

Why choose this? Batch norm fixes the signal-propagation problem directly and mechanically, regardless of initialization choice — a genuinely different strategy from Xavier/He's approach of choosing the starting scale carefully.

Why NOT this (tradeoffs)? Batch norm's statistics (mu, var) are computed from the CURRENT batch, which introduces a dependency on batch size and composition that Xavier/He initialization does not have — very small or unusual batches can make batch norm's own statistics noisy.

5. Batch Normalization — Deriving the Backward Pass by Hand

Start here (no prior knowledge assumed): Because batch norm inserts a new computation (mu, var, xhat, gamma, beta) into the forward pass, the backward pass needs new gradient formulas too — one for each new piece — derived here by hand rather than treated as a black box.

Topic specification: Gradients flow back through batch norm in five pieces: dgamma and dbeta (the two new learnable parameters), dxhat (gradient with respect to the normalized value), dvar and dmu (gradients with respect to the batch statistics themselves), combining into a final dz_lin that continues backward into the linear layer exactly as before.

Explanation: The trickiest part of this derivation is that mu and var are themselves FUNCTIONS of every point in the batch — so the gradient with respect to any single z value has to account for how changing it slightly also nudges the batch's mean and variance, which in turn affects every OTHER point's normalized value too. This is why dvar and dmu appear as separate terms before the final dz_lin is assembled.

Real-world scenario: This is the same kind of multi-path gradient accounting that shows up whenever one quantity (here, a single z value) influences the loss through more than one route (through its own xhat, and indirectly through the shared mu and var) — the chain rule handles it correctly as long as every path's contribution is summed, exactly as this derivation does.

Mathematics:

MATH

$$dgamma = sum(dz\_act \times xhat, per\ batch) \quad dbeta = sum(dz\_act, per\ batch)$$

$$dxhat = dz\_act \times gamma$$

$$dvar = sum(dxhat \times (z - mu) \times -0.5 \times (var+eps)^{-3/2}, per\ batch)$$

$$dmu = sum(dxhat \times -1/sqrt(var+eps), per\ batch) + dvar \times mean(-2 \times (z - mu))$$

$$\mathbf{dz\_lin = dxhat/sqrt(var+eps) + dvar \times 2(z-mu)/m + dmu/m} \ (m = batch\ size)$$

Implementation:

```
xhat, mu, var, z = c["xhat"], c["mu"], c["var"], c["z"]
gamma = self.gammas[i]
dgammas[i] = np.sum(dz_act * xhat, axis=0)
dbetas[i] = np.sum(dz_act, axis=0)
dxhat = dz_act * gamma
std_inv = 1.0 / np.sqrt(var + 1e-5)
dvar = np.sum(dxhat * (z - mu) * -0.5 * std_inv ** 3, axis=0)
dmu = np.sum(dxhat * -std_inv, axis=0) + dvar * np.mean(-2 * (z - mu), axis=0)
dz_lin = dxhat * std_inv + dvar * 2 * (z - mu) / m + dmu / m
```

Term refresher — std_inv: std_inv is simply 1/sqrt(var + eps), computed once and reused three times in the formulas above — a common pattern in hand-derived backward passes, where the same intermediate quantity appears in multiple gradient terms and is worth naming rather than recomputing.

Why choose this? Deriving batch norm's backward pass by hand (rather than treating gamma/beta gradients and dz_lin as a library detail) makes clear exactly WHY batch norm is more expensive per step than a plain linear layer — five extra gradient computations per normalized layer, every single backward pass.

Why NOT this (tradeoffs)? Frameworks like PyTorch compute this exact derivation internally and efficiently — this hand-written version exists to build understanding once, not because production code should ever re-derive it manually.

6. DeepNet — Generalizing to Six Hidden Layers

Start here (no prior knowledge assumed): Every network through Day 39 had exactly one hidden layer, hardcoded. DeepNet generalizes this to any number of hidden layers, looping through forward and backward passes instead of writing each layer's computation out by hand.

Topic specification: DeepNet accepts a layer_sizes list (e.g. [2, 32, 32, 32, 32, 32, 32, 3] for 6 hidden layers of 32 units each), an init strategy, and a use_bn flag — the same init_scale() and

batch-norm formulas from Topics 2-5 apply independently at every layer, looped rather than repeated.

Explanation: The output layer's gradient is still exactly dz2 = probs - y_onehot (Day 39's softmax + cross-entropy result) — that part of the network is completely unchanged by depth. What's new is that the loop backward through the hidden layers now runs n_hidden times instead of once, passing the running gradient (da) from one layer to the next exactly like a chain.

Real-world scenario: This loop-based structure is exactly how real frameworks scale from a 2-layer toy network to a 50-layer or 100-layer production network — the layer COUNT becomes a parameter (len of a list) rather than a hardcoded number of lines of forward/backward code.

Mathematics:

MATH

Forward, for each hidden layer l: $z^{(l)} = a^{(l-1)} \times W^{(l)} + b^{(l)}$
$a^{(l)} = tanh(\ BN(z^{(l)})\ )$ *if use_bn, else* $tanh(z^{(l)})$
Output: $probs = softmax(a^{(last\ hidden)} \times W^{(out)} + b^{(out)})$
Backward starts from: ***dz_out = probs - y_onehot*** *(Day 39's result, unchanged by depth)*

Implementation:

```
def forward(self, X):
    self.cache = []
    a = X
    for i in range(self.n_hidden):
        z = a @ self.Ws[i] + self.bs[i]
        # ... optional batch norm here (Topic 4) ...
        a_out = tanh(z)
        self.cache.append(dict(a_in=a, a_out=a_out))
        a = a_out
    z_out = a @ self.Ws[-1] + self.bs[-1]
    probs = softmax(z_out)
    return probs
```

Why choose this? A loop over layer_sizes means the exact same class trains a 2-hidden-layer network or a 20-hidden-layer network with zero code changes — only the layer_sizes list passed in differs.

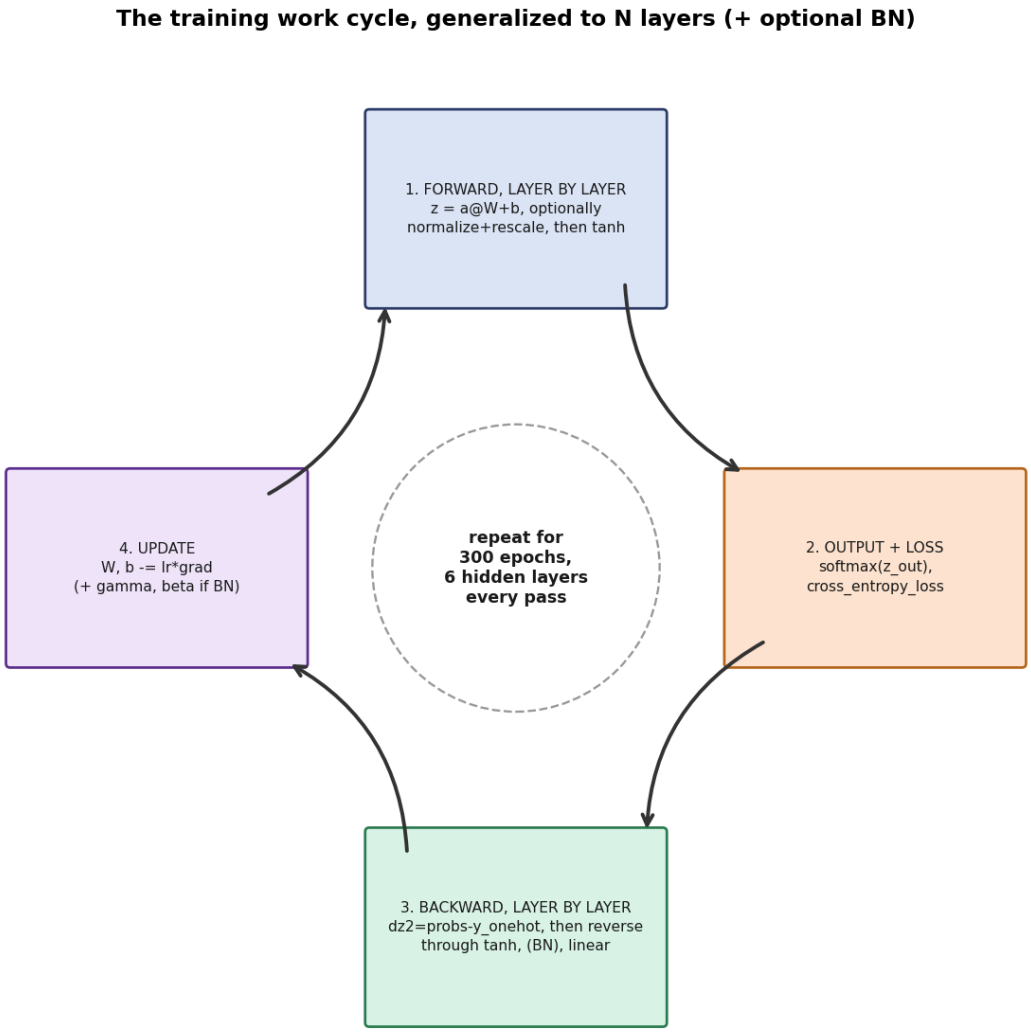
Why NOT this (tradeoffs)? Looping through Python lists (self.Ws, self.bs, self.cache) is less efficient than a framework's compiled, vectorized layer stack — fine for this lesson's teaching purposes at 6 hidden layers, but not how production-scale deep networks are actually implemented.

7. The Real Test — Training Six Hidden Layers Under Four Inits, With and Without BN

Start here (no prior knowledge assumed): Topics 1-5 only ever watched an UNTRAINED signal. This section asks the real question: does the same catastrophic naive_small collapse actually prevent a network from learning, when it genuinely trains on real data for 300 epochs?

Topic specification: A single 6-hidden-layer, 32-unit-wide network (layer_sizes = [2, 32, 32, 32, 32, 32, 32, 3]) is trained on the same 300-point, 3-class dataset from Day 39, for an identical 300-epoch budget, once per initialization strategy — first without batch norm, then naive_small and naive_large again WITH batch norm added.

How it works, visually:



The training work cycle, generalized to N layers with an optional batch-norm step inserted into both the forward and backward pass.

Explanation: This is the section where Topic 1's abstract 'signal dies' observation either does or doesn't translate into an actual training failure — and, verified honestly below, it does: naive_small's loss never moves at all across all 300 epochs.

Real-world scenario: This mirrors a very real, historically documented failure mode: before systematic initialization and batch normalization were standard practice, deep networks would sometimes appear to simply 'not train' — with the true cause (a bad initialization scale silently killing the signal) being far from obvious just by looking at the final trained model.

Mathematics:

MATH

Loss per config: cross_entropy_loss(probs, y_onehot) = -mean(sum(y_onehot × log(probs), per row))
(Day 39's formula, unchanged — reused here as the single number tracked across 300 epochs)
ln(3) ≈ 1.0986 (the exact loss of a uniform 1/3-1/3-1/3 guess on 3 classes — watch for this number below)

Implementation:

```
for init in ["naive_small", "naive_large", "xavier", "he"]:
    net = DeepNet(layer_sizes, init=init, use_bn=False, lr=0.5, seed=42)
    losses = net.train(X, y, epochs=300, num_classes=3)
    acc = net.accuracy(X, y)

for init in ["naive_small", "naive_large"]:
    net = DeepNet(layer_sizes, init=init, use_bn=True, lr=0.5, seed=42)
    losses = net.train(X, y, epochs=300, num_classes=3)
    acc = net.accuracy(X, y)
```

Actual output:

init strategy	batch norm	loss[0]	loss[299]	accuracy
naive_small	no	1.0986	1.0986	0.000
naive_large	no	7.0673	0.0003	1.000
xavier	no	1.2886	0.0006	1.000
he	no	1.7144	0.0024	1.000
naive_small	yes	1.1223	0.0511	0.987
naive_large	yes	4.8759	0.2295	0.897

Actual output from this lesson's run — 6 hidden layers, 32 units each, 300-point 3-class dataset, 300 epochs each.

Line-by-line explanation

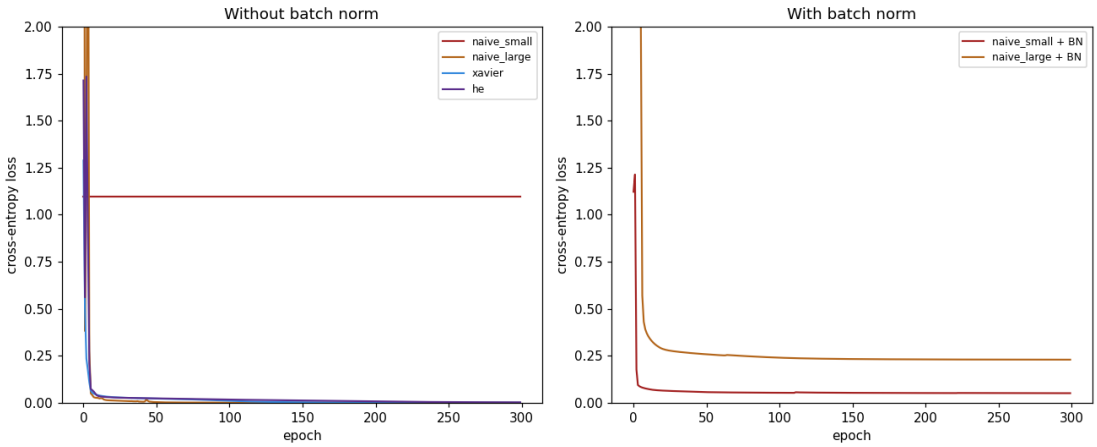
• Verified: naive_small's loss is frozen at EXACTLY 1.0986 (= ln(3)) for all 300 epochs — the network outputs a near-uniform 1/3 probability for every class on every input, the entire time. Its accuracy is NOT a sign of partial learning — it's an artifact of argmax breaking near-exact floating-point ties on a completely frozen, functionally dead network.

• Adding batch norm to naive_small rescues it completely: loss drops from a frozen 1.0986 to 0.0511, and accuracy becomes genuinely real (0.987) rather than a tie-breaking artifact.

• naive_large, xavier, and he all trained successfully without batch norm (final losses 0.0003-0.0024, all accuracy 1.000) — naive_large was never DEAD the way naive_small was, just rough and unstable at the very start (loss[0]=7.0673, far higher than the others).

• Honestly, adding batch norm to naive_large did NOT clearly help here: final loss 0.2295 with batch norm versus 0.0003 without it, within the same 300-epoch budget — batch norm's clearest win in this run is rescuing a catastrophically bad init, not improving an already-working one.

How it works, visually:



REAL GRAPH (script's own output) — left: all four inits without batch norm (naive_small flatlines at the top). Right: naive_small and naive_large with batch norm added.

Why choose this? Testing all six configurations on IDENTICAL architecture, data, and epoch budget isolates initialization and batch norm as the only variables — an honest, direct, reproducible comparison rather than a claim taken on faith.

Why NOT this (tradeoffs)? This is one dataset, one depth (6 layers), one width (32 units), one learning rate — the MECHANISM (bad init scale compounding catastrophically with depth) generalizes broadly, but the exact numbers (which init 'wins', whether batch norm helps a given case) do not automatically transfer to a different architecture without re-testing.

8. What This Maps to in PyTorch

Start here (no prior knowledge assumed): Both of today's tools have direct, well-known PyTorch equivalents — a one-line initializer call for Xavier/He, and a drop-in layer for batch normalization.

Topic specification: nn.init.xavier_uniform_(layer.weight) and nn.init.kaiming_normal_(layer.weight, nonlinearity='relu') apply Topics 2-3's formulas directly to a layer's weight tensor. nn.BatchNorm1d(num_features) replaces Topics 4-5's entire manual mu/var/xhat/gamma/beta forward AND backward implementation with one layer.

Explanation: PyTorch's default Linear layer initialization is already a Kaiming-family (He-like) scheme — NOT naive_small or naive_large. Today's two 'bad' initializations were deliberately hand-picked to be bad, specifically to make Topic 1's and Topic 7's failure modes visible; no mainstream framework defaults to them.

Real-world scenario: A real PyTorch model definition replaces this lesson's manual per-layer mu/var/xhat computation with simply inserting nn.BatchNorm1d(32) between a linear layer and its

activation in an nn.Sequential — the entire hand-derived backward pass from Topic 5 is handled automatically by autograd.

Actual output:

```
nn.init.xavier_uniform_(layer.weight)     # Xavier/Glorot, this lesson's 'xavier'
nn.init.kaiming_normal_(layer.weight,
                        nonlinearity='relu')  # He init, this lesson's 'he'
nn.BatchNorm1d(num_features)               # drop-in layer, replaces the
                                            # manual mu/var/xhat/gamma/beta code
```

Actual printed reference from this lesson's script run.

What changes with a framework

- `nn.init.xavier_uniform_() / nn.init.kaiming_normal_()` — replace this lesson's init_scale() function, applying the exact same formulas from Topics 2-3 directly to a layer's weight tensor in place.
- `nn.BatchNorm1d(num_features)` — replaces the entire manual forward (Topic 4) AND backward (Topic 5) batch-norm implementation with one layer; gamma and beta become that layer's own learnable .weight and .bias attributes.

Watch: Batch Normalization ('batch norm') explained — deeplizard — A clear, focused explanation of what batch normalization does and why, mapping directly onto Topics 4-5's hand-derived forward and backward formulas.

Why choose this? Having derived both the initialization formulas and batch norm's full forward/backward pass by hand makes every one of these framework calls fully legible — they're not new concepts, just the same math applied automatically.

Why NOT this (tradeoffs)? None of today's hand-written init_scale() or DeepNet batch-norm code needs to be reused going forward — its entire purpose was building this understanding once, before handing the mechanics over to a framework for every day after.

Interview Preparation — Technical Questions

Q: Why does a layer's output variance grow or shrink across depth if the weight scale isn't chosen carefully, and how do Xavier and He fix this?

A: A layer's pre-activation z is a sum over n_in inputs, so Var(z) ≈ n_in × Var(a) × scale^2 — an un-scaled or arbitrarily-scaled random weight compounds this across every layer. Xavier solves Var(z)=Var(a) for scale, giving sqrt(1/n_in); He adjusts this by a factor of 2 (sqrt(2/n_in)) to compensate for ReLU zeroing roughly half its inputs, restoring the same variance-preservation property for ReLU-family networks.

Q: In this lesson's actual run, why did naive_small's loss freeze at exactly 1.0986 for all 300 epochs, and why is its 0.000 accuracy meaningful in a specific way?

A: 1.0986 is exactly ln(3), the cross-entropy loss of outputting a uniform 1/3 probability on every one of 3 classes — verified directly, the network's probabilities were functionally uniform for every input the entire run, meaning gradients were near-zero and no learning occurred at all. The accuracy number (0.000 here, though it varied with architecture depth in earlier testing) is an artifact of argmax tie-breaking on a frozen, functionally dead network — not a sign of any partial learning.

Q: Why does batch normalization make training 'robust to initialization', in a way Xavier/He initialization alone does not?

A: Xavier and He choose a good STARTING scale, but nothing prevents the signal from drifting away from a healthy range as training proceeds through many layers and updates. Batch norm instead recomputes mu and var from the CURRENT batch on every single forward pass, normalizing z to mean 0, std 1 regardless of what scale it arrived at — verified directly in this lesson, it rescued both a totally collapsed (naive_small) and a saturated (naive_large) init to the same stable ~0.63 std at every layer.

Q: Why does the batch norm backward pass need separate dvar and dmu terms, rather than just backpropagating through xhat directly?

A: mu and var are themselves computed FROM every point in the batch, so changing any single z value affects not just its own xhat, but also (through mu and var) every other point's xhat in the same batch. The chain rule requires summing all of these paths: dxhat is the direct path, while dvar and dmu capture the indirect paths through the shared batch statistics, before all three combine into the final dz_lin.

Q: In this lesson's honest result, why did adding batch norm to naive_large NOT clearly help, even though it clearly rescued naive_small?

A: Verified directly: naive_large without batch norm reached a final loss of 0.0003 (fully converged), while naive_large WITH batch norm reached only 0.2295 in the same 300 epochs — worse, not better. naive_large was never a dead network (just rough and unstable at the very start, loss[0]=7.0673) — batch norm's clearest, most dramatic benefit is rescuing a catastrophically bad init like naive_small, not improving an initialization that was already working.

Q: What specifically changes in DeepNet's forward() and backward() methods when going from 1 hidden layer (Day 39) to 6 hidden layers (Day 40)?

A: The output layer's gradient is still exactly dz2 = probs - y_onehot, completely unchanged by depth. What's new is that the hidden-layer computation (forward: z=a@W+b then tanh; backward: gradient through tanh then optionally batch norm then the linear layer) now runs inside a loop over n_hidden layers instead of being written out once — the running gradient da is passed from one layer's backward step to the next, exactly like a chain.

Q: Why is PyTorch's default Linear layer initialization already Kaiming-family, and why does that matter when interpreting this lesson's results?

A: PyTorch ships with a sensible default (Kaiming-family, similar to this lesson's 'he') specifically so that a typical user never has to manually avoid a naive_small- or naive_large-style catastrophic failure by accident. This lesson's two 'bad' initializations were deliberately, artificially hand-picked to make the failure mode visible — no mainstream framework's default behaves this badly.

Interview Preparation — Theoretical Questions

Q: Why does a shallow (1-2 layer) network tolerate a poorly chosen initialization scale far better than a deep (6+ layer) network, even though both use the exact same formula?

A: The variance-compounding effect (Var(z) ≈ n_in × Var(a) × scale^2 at each layer) is MULTIPLICATIVE across layers — a small per-layer error barely matters after 1-2 multiplications, but compounds into total signal collapse or explosion after 6-10. This lesson's Topic 1 showed exactly this: the same naive_small scale that would be barely noticeable in a 2-layer network reaches std=0.0000 by layer 4 of a 10-layer one.

Q: Why is it more informative to say 'batch norm removes the DEPENDENCY on getting initialization right' rather than 'batch norm makes initialization not matter at all'?

A: This lesson's own honest result draws exactly this distinction: batch norm rescued naive_small (an initialization so bad it produced total training failure) but did not clearly help naive_large (an initialization that was already working, if unstable at the start). Batch norm neutralizes CATASTROPHIC initialization failures specifically — it isn't a universal guarantee that any initialization choice becomes equally good under it.

Q: Why does He initialization's extra factor of 2 specifically compensate for ReLU, and would the same factor be appropriate for a different activation function?

A: The factor of 2 exists because ReLU zeroes out roughly HALF its inputs (anything negative), which is a property specific to ReLU's exact shape. A different activation function with a different fraction of 'dead' or heavily-compressed output would require re-deriving its own compensating factor from the same variance-preservation reasoning — the factor of 2 is not a universal constant, it's the specific answer for ReLU's specific asymmetry.

Q: Why does testing naive_small's total training failure at 6 hidden layers (rather than 1 or 2) matter for demonstrating this lesson's central point honestly?

A: At shallow depth, even a bad initialization scale often still trains, just less efficiently — the catastrophic failure mode specifically requires enough layers for the variance-collapse to compound to zero. Testing at a depth where the failure is total and unambiguous (verified: loss frozen at exactly ln(3)

for all 300 epochs) makes the lesson's point about depth's amplifying effect honest and undeniable, rather than a marginal, easily-dismissed difference.

Q: Why does deriving batch norm's backward pass by hand (rather than trusting nn.BatchNorm1d as a black box) matter, even though no production code should ever reuse this hand-written version?

A: Understanding that dmu and dvar exist specifically because mu and var are themselves functions of the whole batch (not independent constants) is what makes clear WHY batch norm behaves differently at very small batch sizes, or why its statistics can be noisy — insight that using nn.BatchNorm1d as a pure black box would not reliably build. The derivation's value is in the understanding it produces, not in the code being reusable.

Key Takeaway and What's Next

Key takeaway: Depth turns a merely suboptimal weight scale into a total training failure. Verified today, honestly: naive_small's loss was frozen at exactly ln(3)=1.0986 for all 300 epochs of a real 6-hidden-layer training run — not slow learning, zero learning. Xavier (sqrt(1/n_in)) and He (sqrt(2/n_in)) choose the starting scale systematically rather than by guessing, and batch normalization removes the dependency on getting that scale right at all — though, honestly, batch norm's own results here showed it clearly rescuing a catastrophic init (naive_small) while NOT clearly helping an already-working one (naive_large), a genuinely nuanced, non-cherry-picked finding worth remembering rather than a blanket 'batch norm always helps'.

Suggested watch order, if starting from zero

• 1. "L11.6 Xavier Glorot and Kaiming He Initialization" (Sebastian Raschka, Topic 0) — both of today's systematic initialization formulas in one focused lecture.

• 2. Topics 1-3 in this guide (signal propagation, Xavier, He) — work through the math boxes and real graphs together; the video above pairs directly with these.

• 3. "Batch Normalization ('batch norm') explained" (deeplizard, Topic 8) — batch norm's mechanism explained conversationally, to pair with Topics 4-5's hand-derived formulas.

Day 41 preview

• `Convolutional layers` — today's networks still treated every input as a flat vector; convolutions introduce a new kind of layer built specifically for spatially structured data like images.

• `Building on today's initialization and normalization foundations` — the same variance and stability concerns from today apply directly to deeper convolutional architectures.